Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# 2)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: **AHMED JAVED QURESHI** Roll No: **24i-6554** Section: **CE-A**

Date performed: **25 August, 2026**

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task I wrote Verilog codes for Half Adder and Full Adder using both Gate level modeling and also for Data flow modeling. I also then designed Half Adder and Full Adder on Logisim. I also used testbench to check all the possible inputs.

Code:

HALF ADDER:

GATE LEVEL:

```
module HalfAdder(a,b,Sum,Carry);

input a;
input b;
output Sum;
output Carry;

xor x1( Sum,a,b);
and a1(Carry ,a,b);

endmodule

module testbench();

reg x;
reg y;

wire S;
wire C;

HalfAdder uut(x,y,S,C);

initial
begin

x=0; y=0;
#50
x=0; y=1;
```

```
#50
x=1; y=0;
#50
x=1;y=1;

end
endmodule
```

DATA FLOW:

```
module HalfAdder(a,b,Sum,Carry);

input a;
input b;
output Sum;
output Carry;

assign Sum = a^b;

assign Carry = a&b;


endmodule

module testbench();

reg x;
reg y;

wire S;
wire C;

HalfAdder uut(x,y,S,C);

initial
begin

x=0; y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
```

```
x=1;y=1;

end
endmodule
```

FULL ADDER:

GATE LEVEL:

```
module FullAdder(a,b,Cin,Sum,Carry);

input a;
input b;
input Cin;
output Sum;
output Carry;

wire w1;
wire z1;
wire z2;

xor x1(w1,a,b);
xor x2(Sum,w1,Cin);

and a1(z1,a,b);
and a2(z2,w1,Cin);

or o1(Carry,z1,z2);

endmodule


module testbench();

reg x;
reg y;
reg Cin;

wire S;
wire C;
```

```
FullAdder uut(x,y,Cin,S,C);

initial
begin

x=0; y=0; Cin=0;
#50
x=0; y=0; Cin=1;
#50
x=0; y=1; Cin=0;
#50
x=0; y=1; Cin=1;
#50
x=1; y=0; Cin=0;
#50
x=1; y=0; Cin=1;
#50
x=1; y=1; Cin=0;
#50
x=1; y=1; Cin=1;
#50;

end

endmodule
```

DATA FLOW:

```
module FullAdder(a,b,Cin,Sum,Carry);

input a;
input b;
input Cin;
output Sum;
output Carry;

assign Sum = a ^ b ^ Cin;

assign Carry = (a & b) | (a & Cin) | (b & Cin);
```

```
endmodule


module testbench();

reg x;
reg y;
reg Cin;

wire S;
wire C;

FullAdder uut(x,y,Cin,S,C);

initial
begin

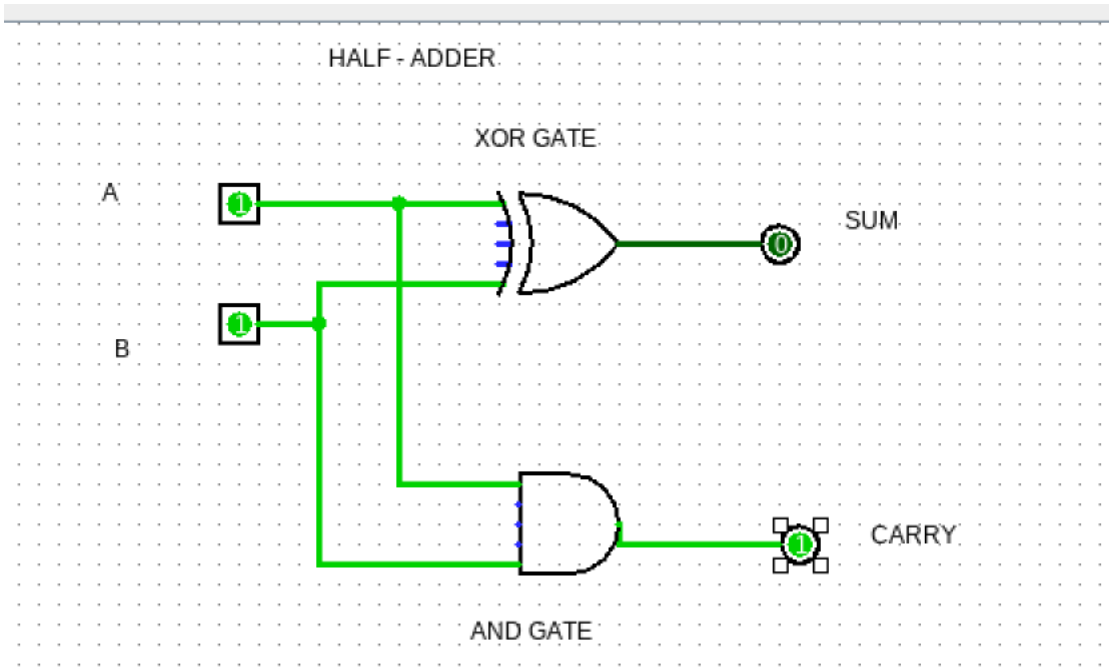
x=0; y=0; Cin=0;
#50
x=0; y=0; Cin=1;
#50
x=0; y=1; Cin=0;
#50
x=0; y=1; Cin=1;
#50
x=1; y=0; Cin=0;
#50
x=1; y=0; Cin=1;
#50
x=1; y=1; Cin=0;
#50
x=1; y=1; Cin=1;
#50;

end

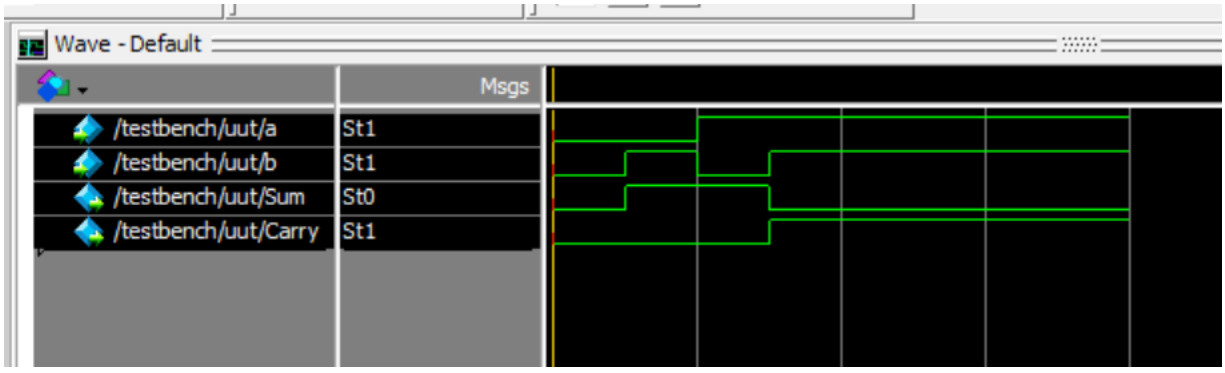
endmodule
```

Output:

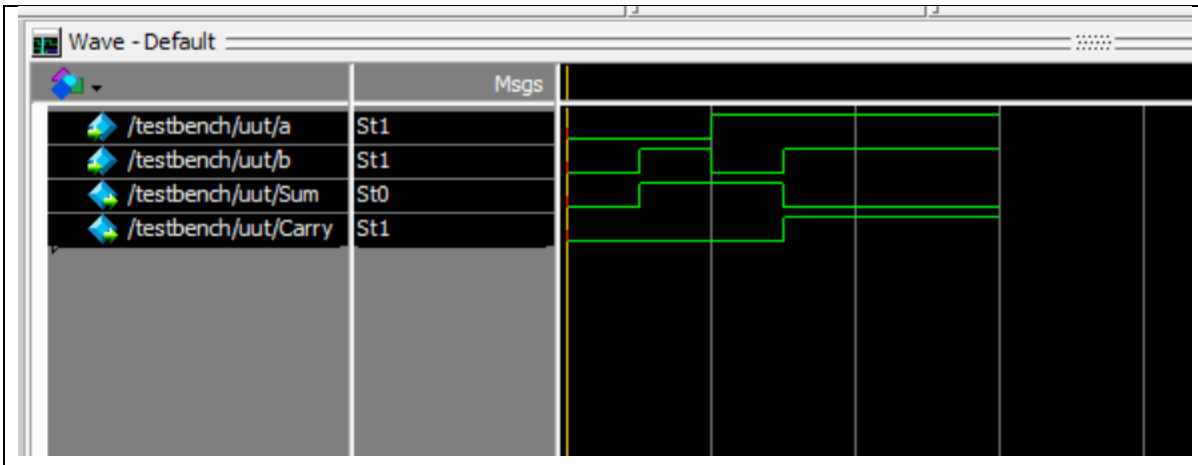
HALF ADDER:



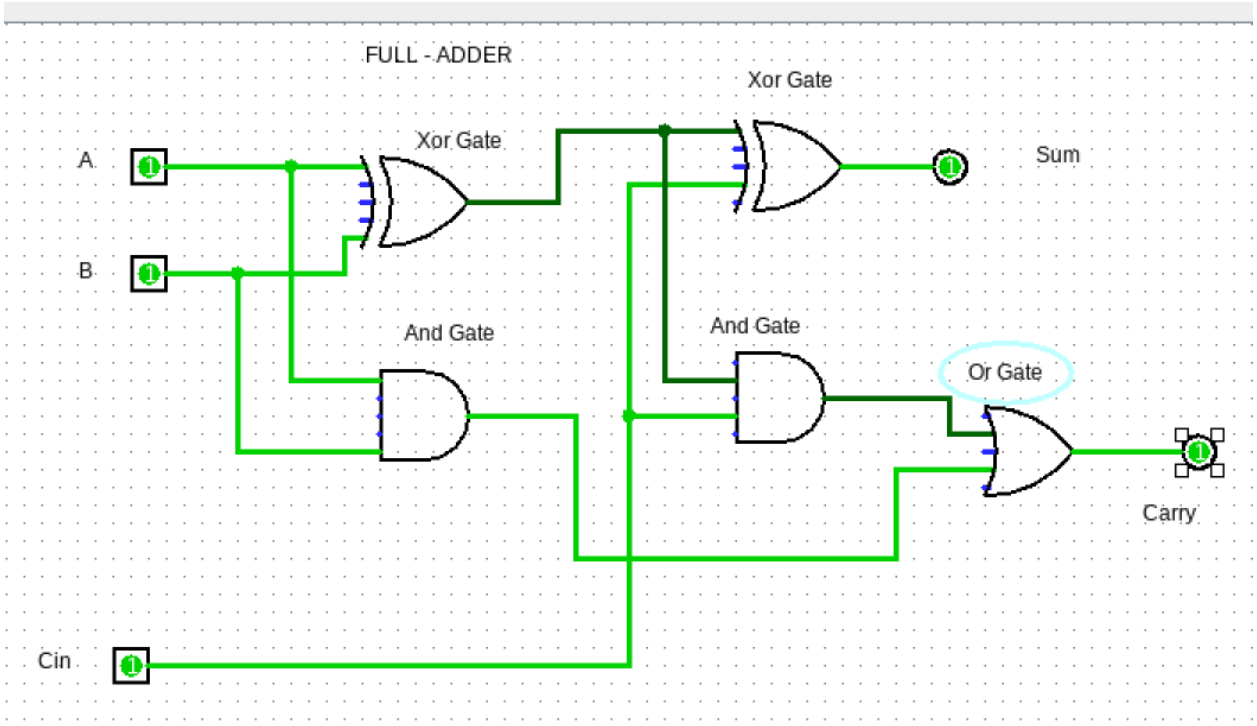
GATE LEVEL:



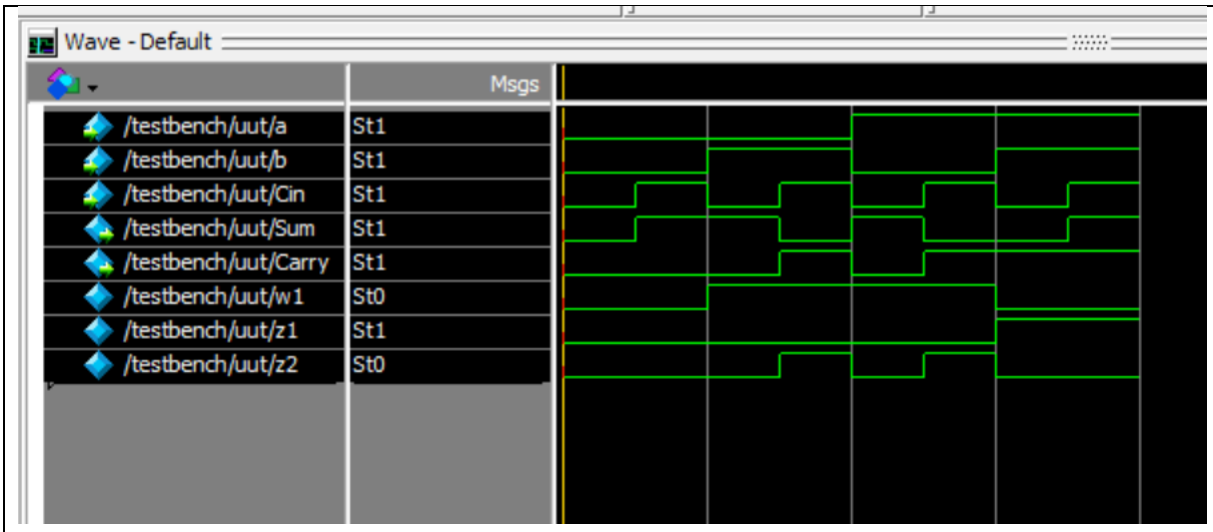
DATA FLOW:



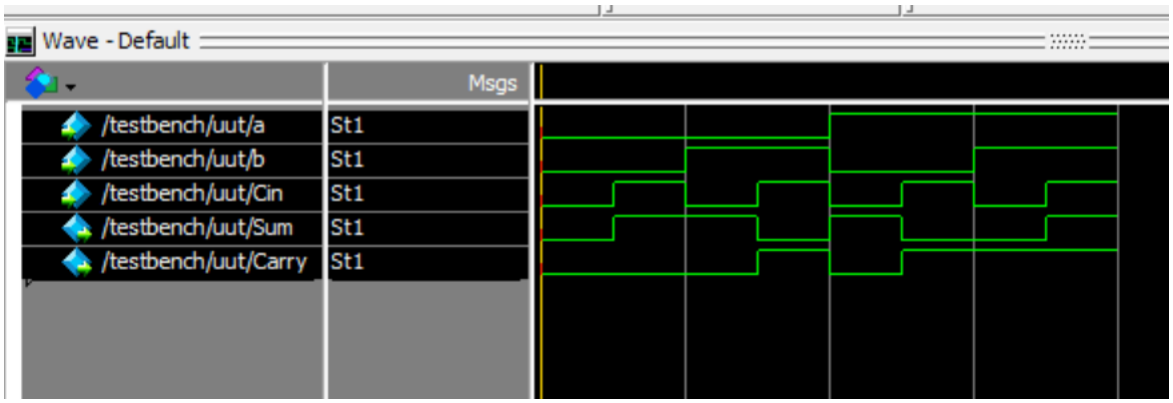
FULL ADDER:



GATE LEVEL:


Wave - Default
Msgs
/testbench/uut/a
/testbench/uut/b
/testbench/uut/Cin
/testbench/uut/Sum
/testbench/uut/Carry
/testbench/uut/w1
/testbench/uut/z1
/testbench/uut/z2
St1
St1
St1
St1
St1
St0
St1
St0

DATA FLOW:


Wave - Default
Msgs
/testbench/uut/a
/testbench/uut/b
/testbench/uut/Cin
/testbench/uut/Sum
/testbench/uut/Carry
St1
St1
St1
St1
St1

Task 2:

Description:
In this task, I designed a 3×8 decoder in Verilog .It takes 3 inputs and gives 8 outputs. I wrote codes for both Gate level and Data flow modeling. A testbench was also used to check all possible inputs. The simulation gave the expected output for each input combination.
Code:

GATE LEVEL:

```
module Decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);



input a;

input b;

input c;



output y0;

output y1;

output y2;

output y3;

output y4;

output y5;

output y6;

output y7;
```

```
wire na;

wire nb;

wire nc;



not n1(na,a);

not n2(nb,b);

not n3(nc,c);



and a1(y0,na,nb,nc);

and a2(y1,na,nb,c);

and a3(y2,na,b,nc);

and a4(y3,na,b,c);

and a5(y4,a,nb,nc);

and a6(y5,a,nb,c);

and a7(y6,a,b,nc);

and a8(y7,a,b,c);



endmodule




module testbench();
```

```
reg x;

reg y;

reg z;



wire o0;

wire o1;

wire o2;

wire o3;

wire o4;

wire o5;

wire o6;

wire o7;



Decoder uut(x,y,z,o0,o1,o2,o3,o4,o5,o6,o7);



initial

begin



x=0; y=0; z=0;

#50

x=0; y=0; z=1;
```

```
#50

x=0; y=1; z=0;

#50

x=0; y=1; z=1;

#50

x=1; y=0; z=0;

#50

x=1; y=0; z=1;

#50

x=1; y=1; z=0;

#50

x=1; y=1; z=1;



end



endmodule
```

DATA FLOW:

```
module Decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);



input a;

input b;

input c;
```

```
output y0;

output y1;

output y2;

output y3;

output y4;

output y5;

output y6;

output y7;



assign y0=~a&~b&~c;

assign y1=~a&~b&c;

assign y2=~a&b&~c;

assign y3=~a&b&c;

assign y4=a&~b&~c;

assign y5=a&~b&c;

assign y6=a&b&~c;

assign y7=a&b&c;



endmodule
```

```
module testbench();



reg x;

reg y;

reg z;



wire o0;

wire o1;

wire o2;

wire o3;

wire o4;

wire o5;

wire o6;

wire o7;



Decoder uut(x,y,z,o0,o1,o2,o3,o4,o5,o6,o7);



initial

begin



x=0; y=0; z=0;

#50
```

```
x=0; y=0; z=1;

#50

x=0; y=1; z=0;

#50

x=0; y=1; z=1;

#50

x=1; y=0; z=0;

#50

x=1; y=0; z=1;

#50

x=1; y=1; z=0;

#50

x=1; y=1; z=1;

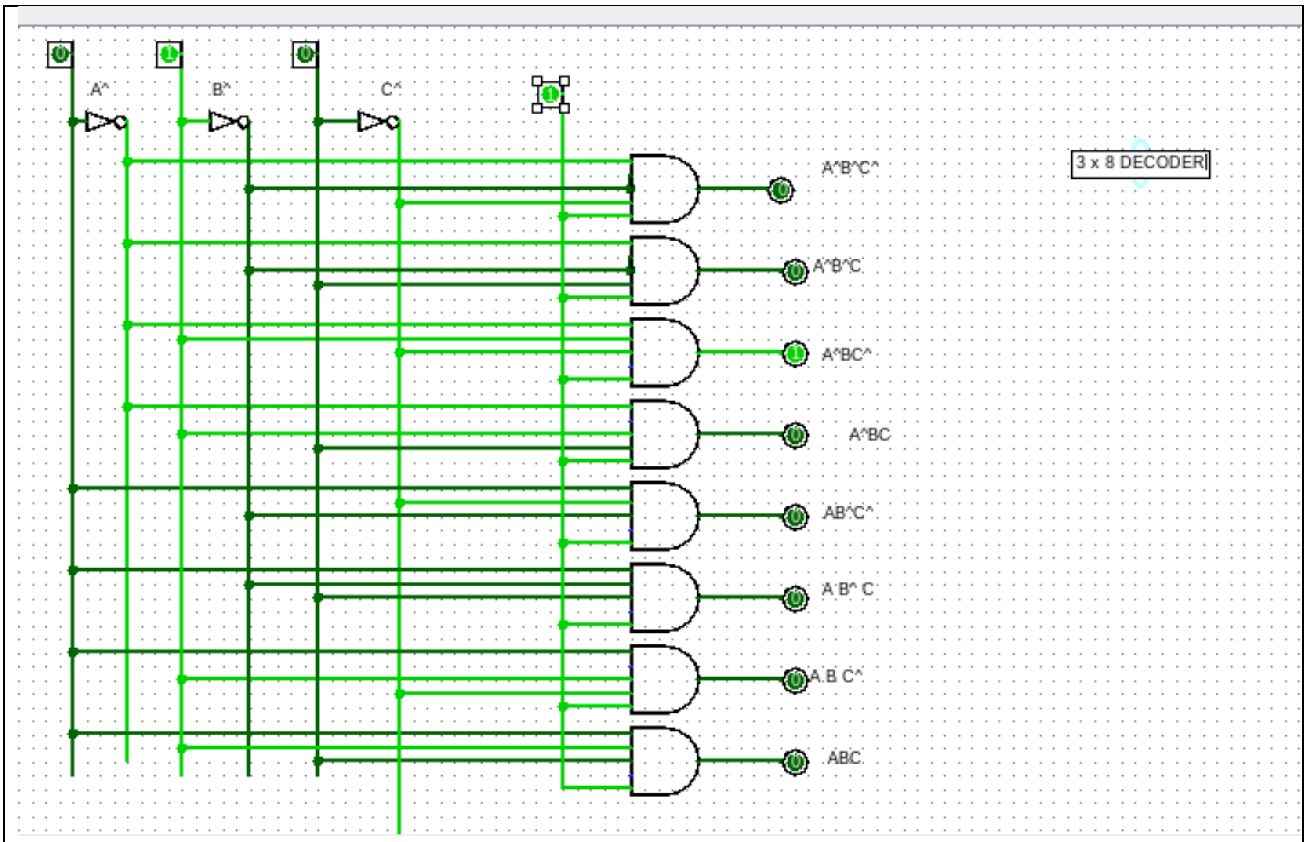


end



endmodule
```

Output:



Task 3:

Description:
In this task, I designed an 8×1 multiplexer in Verilog. It has 8 inputs, 3 select lines, and 1 output. I used both gate level and dataflow modeling. I also used a testbench to test different select combinations. The simulation showed the correct selected input at the output.

Code:

GATE LEVEL:

```
module Mux8to1(a,b,c,d,e,f,g,h,s0,s1,s2,y);



input a;

input b;

input c;

input d;

input e;

input f;

input g;

input h;



input s0;

input s1;

input s2;



output y;



wire ns0;

wire ns1;

wire ns2;
```

```
wire w1;

wire w2;

wire w3;

wire w4;

wire w5;

wire w6;

wire w7;

wire w8;



not n1(ns0,s0);

not n2(ns1,s1);

not n3(ns2,s2);



and a1(w1,a,ns2,ns1,ns0);

and a2(w2,b,ns2,ns1,s0);

and a3(w3,c,ns2,s1,ns0);

and a4(w4,d,ns2,s1,s0);

and a5(w5,e,s2,ns1,ns0);

and a6(w6,f,s2,ns1,s0);

and a7(w7,g,s2,s1,ns0);

and a8(w8,h,s2,s1,s0);



or o1(y,w1,w2,w3,w4,w5,w6,w7,w8);
```

```
endmodule



module testbench();

reg a;
reg b;
reg c;
reg d;
reg e;
reg f;
reg g;
reg h;

reg s0;
reg s1;
reg s2;

wire y;

Mux8to1 uut(a,b,c,d,e,f,g,h,s0,s1,s2,y);
```

```
initial

begin



a=1; b=0; c=0; d=0; e=0; f=0; g=0; h=0;



s2=0; s1=0; s0=0;

#50

s2=0; s1=0; s0=1;

#50

s2=0; s1=1; s0=0;

#50

s2=0; s1=1; s0=1;

#50

s2=1; s1=0; s0=0;

#50

s2=1; s1=0; s0=1;

#50

s2=1; s1=1; s0=0;

#50

s2=1; s1=1; s0=1;



end
```

```
endmodule
```

DATA FLOW:

```
module Mux8to1(a,b,c,d,e,f,g,h,s0,s1,s2,y);



input a;

input b;

input c;

input d;

input e;

input f;

input g;

input h;



input s0;

input s1;

input s2;



output y;



assign y=(a&~s2&~s1&~s0)|

      (b&~s2&~s1&s0)|

      (c&~s2&s1&~s0)|

      (d&~s2&s1&s0)|
```

```
        (e&s2&~s1&~s0)|

        (f&s2&~s1&s0)|

        (g&s2&s1&~s0)|

        (h&s2&s1&s0);


endmodule



module testbench();


reg a;

reg b;

reg c;

reg d;

reg e;

reg f;

reg g;

reg h;


reg s0;

reg s1;

reg s2;
```

```
wire y;



Mux8to1 uut(a,b,c,d,e,f,g,h,s0,s1,s2,y);



initial

begin



a=1; b=0; c=0; d=0; e=0; f=0; g=0; h=0;



s2=0; s1=0; s0=0;

#50

s2=0; s1=0; s0=1;

#50

s2=0; s1=1; s0=0;

#50

s2=0; s1=1; s0=1;

#50

s2=1; s1=0; s0=0;

#50

s2=1; s1=0; s0=1;

#50

s2=1; s1=1; s0=0;

#50
```

```
s2=1; s1=1; s0=1;


end


endmodule
```

Output:

